

## Тема 2 - Формални модели – машина на Тюринг, нормални алгоритми на Марков. Тезис на Чърч.

**Машина на Тюринг** е абстрактно изчислително устройство, описано от английския математик Алън Тюринг през 1936 г.

Тюринг използва тази абстракция, за да даде първото точно определение на понятието алгоритъм (наричано още механична, формална или ефективна процедура). Машината се използва при доказването на основни резултати в компютърните науки, най-вече в областите изчислимост и сложност на алгоритмите, както и в математическата логика.

Основният фокус на работата на Тюринг е разбиването на кода „Енигма“. Енигма е вид машина за шифроване, използвана от германските въоръжени сили през Втората световна война.

## Тема 2 - Формални модели – машина на Тюринг, нормални алгоритми на Марков. Тезис на Чърч.

### Нормални алгоритми на Марков

Алгоритъм на Марков, кръстен на съветския математик Андрей Марков, младши, е система за пренаписване на низове, която използва правила за заместване, за да работи върху низ от символи.

Правилата са последователност от двойки низове, обикновено представени под формата на модел  $\rightarrow$  заместване. Всяко правило може да бъде обикновено или прекратяващо.

Даден е **входен** низ:

1. Проверете правилата в ред отгоре надолу, за да видите дали някой от шаблоните може да бъде намерен във входния низ.
2. Ако не бъде намерен, алгоритъмът спира.
3. Ако бъде намерен един (или повече), използвайте първия от тях, за да замените най-лявата поява на съответстващ текст във входния низ с неговата замяна.
4. Ако току-що приложеното правило е било завършващо, алгоритъмът спира.
5. Отидете на стъпка 1.

## Тема 2 - Формални модели – машина на Тюринг, нормални алгоритми на Марков. Тезис на Чърч.

Следващият пример показва приложението на алгоритъм на Марков.

### Символен низ:

„Аз купих В с А от Т S.“

### Правила

"А" -> „ябълки"

"В" -> „чанта"

"S" -> „магазин"

"Т" -> „кварталния"

„магазин" -> „плод-зеленчук"

„ако не се приложи правило" ->  
„прекрати алгоритъма"

### Изпълнение

Ако алгоритъмът се приложи към горния пример, символният низ ще се промени по следния начин.

"Аз купих В с А от Т S."

"Аз купих В с ябълки от Т S."

"Аз купих чанта с ябълки от Т S."

"Аз купих чанта с ябълки от Т магазин."

"Аз купих чанта с ябълки от кварталния магазин."

"Аз купих чанта с ябълки от кварталния плод-зеленчук."

**Алгоритъмът ще приключи.**

## Тема 2 - Формални модели – машина на Тюринг, нормални алгоритми на Марков. Тезис на Чърч.

### Тезис на Чърч

Формалните модели на всеки един алгоритъм съответстват на интуитивната представа за него.

Тезисът на Чърч по-конкретно твърди, че всички дефиниции на понятието формален метод (алгоритъм, ефективна процедура) са еквивалентни.

Нарича се тезис, защото не е аксиома, не е и математическа теорема, а подобно на физическите закони изразява експериментален опит – всяка нова конструкция на изчисляващо устройство се оказва еквивалентна на коя да е от известните схеми за изчисление (например на машина на Тюринг).

## Тема 2 - Сложност и оценка на алгоритмите.

### Сложност на алгоритмите по време (time complexity).

В компютърните науки времевата сложност е изчислителната сложност, която описва количеството компютърно време, необходимо за изпълнение на алгоритъм.

Времевата сложност обикновено се оценява чрез преброяване на броя на елементарните операции, извършени от алгоритъма, като се предполага, че всяка елементарна операция отнема фиксирано време за изпълнение. По този начин се приема, че количеството отнето време и броят на елементарните операции, извършени от алгоритъма, са свързани с постоянен фактор (**Big O**).

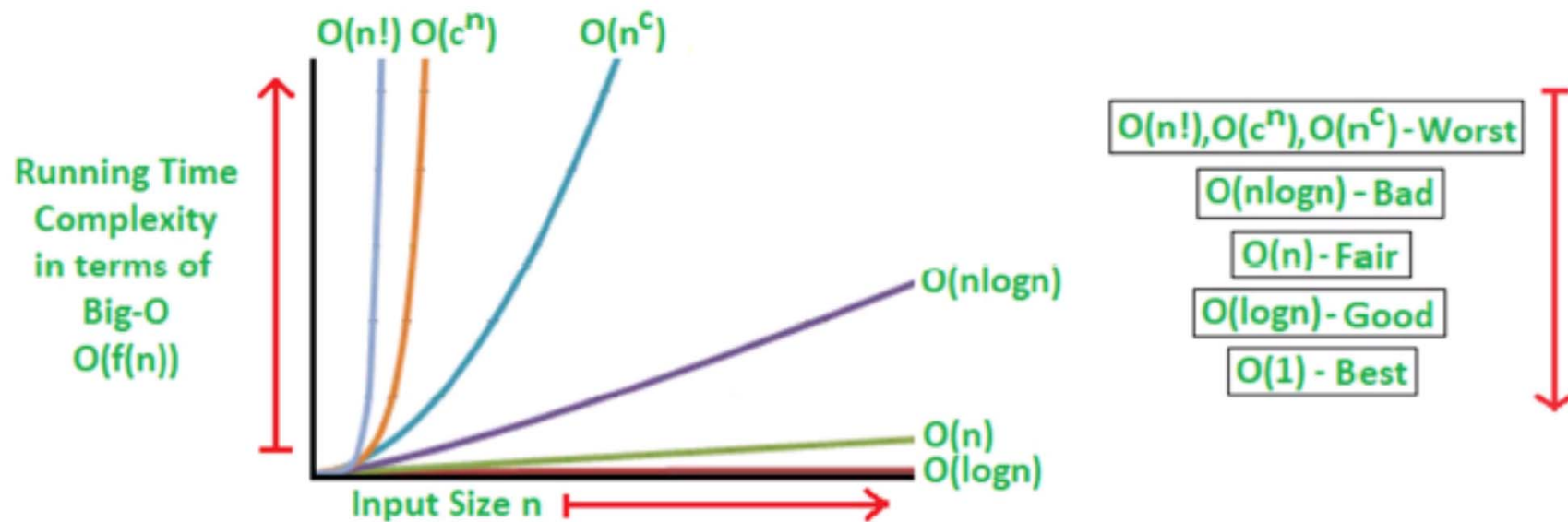
## Тема 2 - Сложност и оценка на алгоритмите.

### Сложност на алгоритмите по време (time complexity).

В компютърните науки нотацията **Big O** се използва за класифициране на алгоритми според това как тяхното време за изпълнение нараства с нарастването на размера на входа.

Нотацията **Big O** характеризира функциите според темповете им на растеж: различни функции със същата асимптотична скорост на растеж могат да бъдат представени с една и съща **Big O** нотация. Буквата **O** се използва, защото скоростта на нарастване на функция също се нарича ред на функцията (**order of the function**). Описание на функция от гледна точка на нотация **Big O** обикновено осигурява само горна граница на скоростта на растеж на функцията.

## Тема 2 - Сложност и оценка на алгоритмите.





## Тема 2 - Сложност и оценка на алгоритмите.

### **Сложност на алгоритмите по време (time complexity).**

Често за решението на една задача може да се използват няколко различни алгоритъма. Някои алгоритми са по-ефективни от други. Оценката за ефективност е свързана с броя на изпълняваните операции върху входните данни, чийто брой се означава с **N**.

**Сложност на алгоритъм** е приблизителна оценка (**O**) на максималния брой операции, необходими за изпълнението на алгоритъм, зависеща от броя на входните данни **N**.



## Тема 2 - Сложност и оценка на алгоритмите.

1. При оценяването на сложност говорим за порядъка на броя операции, а не за техния точен брой.
2. Сложността на алгоритмите се означава най-често с нотацията  $O(f)$ , още позната като асимптотична нотация, където  $f$  е функция на размера (обема) на входните данни.

## Тема 2 - Сложност и оценка на алгоритмите.

### Видове сложност на алгоритмите:

- \* **константна** - означава се  $O(1)$ . За извършване на дадена операция са необходими константен брой стъпки и този брой не зависи от броя на входните данни;
- \* **линейна** – означава се  $O(N)$ . Броят на операциите зависи линейно от броя на входните данни;

Пример:

При намиране на сума на масив с  $N$  елемента, по  $N$  пъти се изпълняват действията:

„добавяне към сумата”; „добавяне на 1 към брояча”; „сравнение”.

Това прави  $3 \cdot N$  броя основни операции, към които може да добавим и няколко константни действия. **Тук зависимостта е линейна.**

## Тема 2 - Сложност и оценка на алгоритмите.

### Видове сложност на алгоритмите:

- \* **квадратична** - означава се  $O(N^2)$ . При обем на входните данни  $N$ , ще се изпълнят операции с основен порядък  $N^2$ .

### Пример:

Ако за два масива с числа от по  $M$  елемента ( $N=2*M$ ), желаем да намерим произведенията на всеки елемент от единия масив с всеки елемент от другия масив, ще са необходими:

$$M_2 = (N/2)^2 \text{ броя основни операции.}$$

## Тема 2 - Сложност и оценка на алгоритмите.

### Видове сложност на алгоритмите:

- \* **логаритмична** - означава се  $O(\log(N))$  или  $O(N \cdot \log(N))$  (линейно логаритмична). Това е относително ниска степен на сложност на алгоритъма, при която броят на входните данни  $N$  е равен на степен на броя на операциите. Най-често използваната основа е 2.
- \* **кубична** – означава се  $O(N^3)$ . За извършване на дадена операция са необходими от порядъка на  $N^3$  стъпки, където  $N$  характеризира обема на входните данни.
- \* **експоненциална** сложност има, когато броят на основните операции се увеличава стремително при увеличение на броя на входните данни. Означения:  $O(N^K)$ ,  $O(K^N)$ ,  $O(N!)$

## Тема 2 - Сложност и оценка на алгоритмите.

### Видове сложност на алгоритмите:

Има алгоритми, чиято сложност зависи от няколко променливи. Ако входните данни са примерно два списъка с различен брой  $M$  и  $N$  на елементите, то всевъзможните двойки комбинации от елементи от двата списъка е  $M*N$ . Тогава сложността на алгоритъм, извеждащ всевъзможните двойки е  $O(M*N)$ .

**Сложността на алгоритмите е прието да се оценява в най-лошия случай, което означава, че в средния случай те могат да работят и по-бързо, но в най-лошия случай няма да работят по-бавно от посочената сложност.**

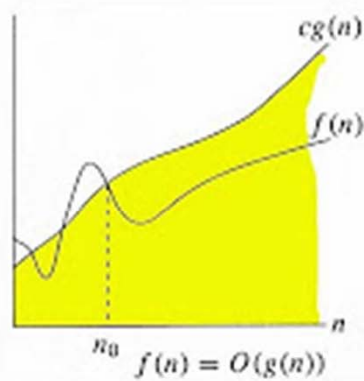
## Тема 2 - Сложност и оценка на алгоритмите.

**Асимптотична нотация** се използва за описание на времето за изпълнение на алгоритъм - колко време отнема алгоритъм с даден вход  $N$ .

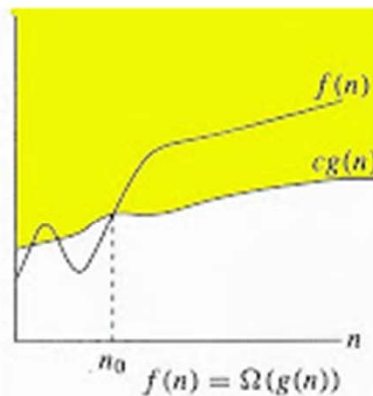
Има три различни обозначения:

- \* **big O** – Дефинира се като горна граница, максималното време, което алгоритъмът ще отнеме (най-лошият случай)
- \* **big Theta ( $\Theta$ )** - Дефинира се като най-строгата връзка и най-строгата връзка е най-доброто от всички най-лоши времена, които алгоритъмът може да отнеме.
- \* **big Omega ( $\Omega$ )** – Дефинира се като долна граница, а долната граница на алгоритъм е най-малкото необходимо време (най-ефективният възможен начин, с други думи най-добрият случай).

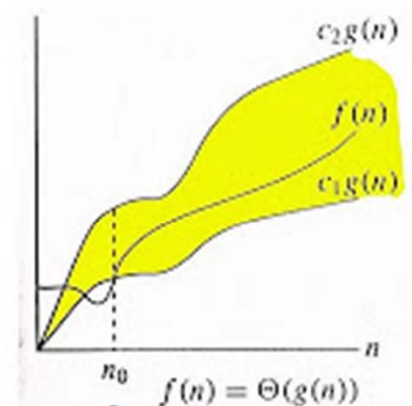
## Тема 2 - Сложност и оценка на алгоритмите.



**Big Oh**



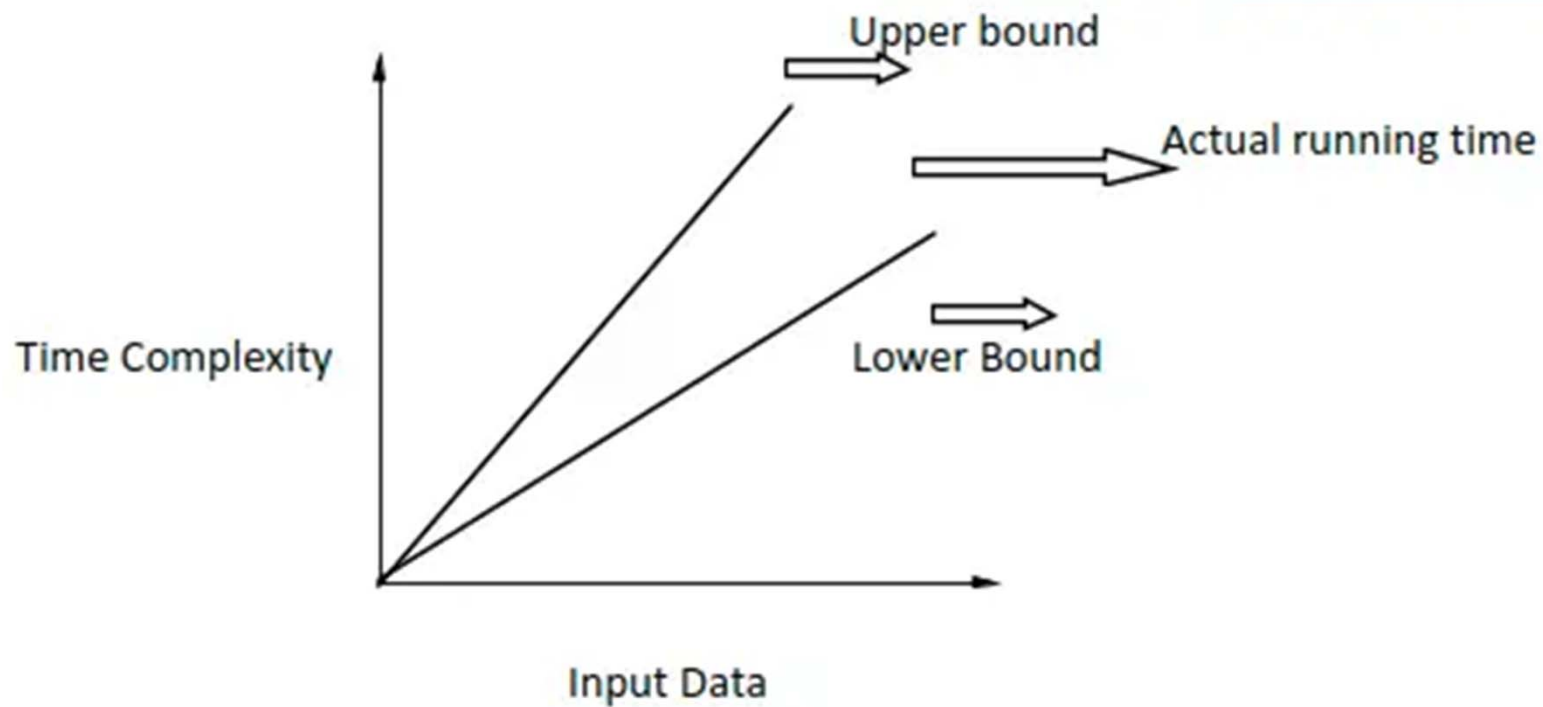
**Omega**



**Theta**



## Тема 2 - Сложност и оценка на алгоритмите.



## Тема 2 - Сложност и оценка на алгоритмите.

**Пространствената сложност (сложността по памет)** на алгоритъм или компютърна програма е количеството памет, необходимо за решаване на екземпляр на изчислителния проблем като функция от характеристиките на входа. Това е паметта, която се изисква от даден алгоритъм, докато не се изпълни докрая.

## Тема 2 - Сложност и оценка на алгоритмите.

Всеки път, когато се пише алгоритъм, решаващ на проблем, е необходима известна памет за неговото изпълнение. За всеки алгоритъм паметта може да се използва за следното:

- \* Променливи (това включва постоянни стойности, временни стойности)
- \* Програмна инструкция
- \* Изпълнение

Пространствената сложност е количеството памет, използвано от алгоритъма (включително входните стойности към алгоритъма), за да го изпълни и произведе резултата. Понякога помощното пространство се бърка с пространствената сложност. Но помощното пространство е допълнителното пространство или временното пространство, използвано от алгоритъма по време на неговото изпълнение.